

REMM: Revisable Embodied Memory for Manipulation

Anonymous Authors

Abstract—Long-horizon manipulation requires robots to retain task-relevant history while revising conclusions no longer supported by physical evidence. Existing memory-augmented controllers largely focus on retaining and retrieving past observations, but updating a changed physical relation alone can leave dependent task conditions stale. We introduce REMM, Revisable Embodied Memory for Manipulation, a supervisory memory layer for a frozen G0.5 vision-language-action executor. REMM separates an append-only evidence history from a revisable task-state representation and records task-semantic dependencies among physical relations, derived scene state, and completed task conditions. When new evidence supersedes a relation, REMM retracts dependent conclusions, identifies the affected task obligation, and issues a bounded recovery instruction at the next stationary skill boundary without updating the executor. We evaluate REMM across 51 physical trials spanning intervention recovery, ordered sequence recall, and hidden-object rearrangement. Relative to the strongest baseline in each task family, REMM improves mean checkpoint progress from 57% to 91% for intervention recovery and from 43% to 70% for sequence recall, while completing all hidden-object trials. In matched offline diagnostics, REMM produces the correct hidden-object choice and grounded command on 30/30 clips, versus at most 11/30 for the baselines. These results show that explicit dependency revision enables changes in physical evidence to drive appropriate subsequent control without retraining the underlying robot policy. Paper website: <https://remm-robot.github.io/>

I. INTRODUCTION

Long-horizon manipulation requires more than executing a sequence of independent skills. Each action changes the world in which subsequent decisions are made, and information needed later may no longer be directly observable: which object was handled first, what remains inside a container, or where an object was placed before becoming occluded. Such tasks therefore require decisions that depend on interaction history under partial observability [1], [2]. Yet retaining history alone is insufficient. A previously valid relation may cease to hold after a slip, human intervention, or environmental rearrangement. A robot must therefore preserve *what happened* while revising *what is currently supported by the available evidence*.

Generalist robot policies have substantially improved the execution of individual manipulation skills [3]–[7]. Recent models further extend language-conditioned control to long-horizon settings; in particular, G0.5 provides an autoregressive interface that interleaves reasoning and action generation within a single policy [7]. However, a capable executor does not by itself determine what should happen next when the relevant evidence is old, hidden, or contradicted by a subsequent physical intervention.

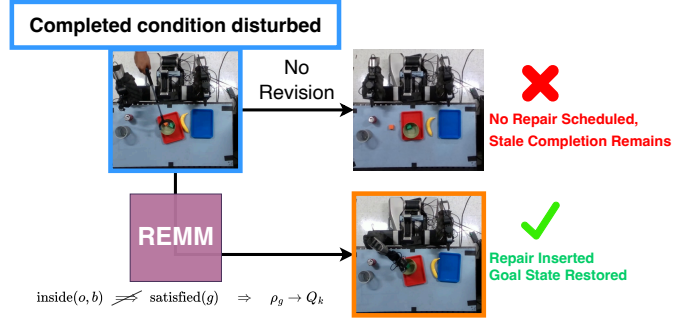


Fig. 1. **Revising memory after physical intervention.** A manipulation history can support either continued execution or recovery depending on whether later evidence preserves or contradicts an earlier task relation. REMM updates the supported task state and changes the subsequent robot action without retraining the frozen executor.

Recent work equips manipulation systems with temporal context through learned memory banks, sparse visual events, or retrieved keyframes [2], [8], [9]. Hierarchical systems instead place memory and reasoning above the motor policy: HiMe [10] maintains editable multimodal memory, while HyMeS [11] and BATON [12] steer reusable or frozen skills using external verification and transition-aware memory. These methods establish that memory can extend a policy beyond its immediate observation. They also sharpen a different question: after evidence changes, which earlier conclusions should change with it?

Closed-loop planners can incorporate success, scene, or human feedback into subsequent decisions [13]; structured world representations encode relational scene state [14]; and dynamic memories can add, delete, or modify stored relations as new evidence arrives [15]. Yet these capabilities do not explicitly track which physical relations support previously completed task conditions. If such a relation changes without revising its downstream consequences, the controller can retain an internally coherent but physically stale task state. We therefore consider an intervention-grounded process memory in which physical relations, derived scene state, and completed task conditions are connected through explicit support dependencies. When verified evidence changes an upstream relation, its affected consequences can be selectively withdrawn. These dependencies are supplied by task and physical semantics rather than learned through causal discovery; their use as recorded justifications is related to truth maintenance [16], while remaining distinct from causal world models that learn or probabilistically infer manipulation outcomes [17], [18].

We introduce **REMM**, **Revisable Embodied Memory for Manipulation**, a supervisory layer around a frozen G0.5 executor. REMM preserves an append-only evidence history while maintaining a separate projection of currently supported state. It records causal dependency links from physical relations to derived and completed task conditions; when verified evidence supersedes a parent relation, REMM invalidates affected descendants, identifies the broken task obligation, and schedules a bounded recovery instruction at the next safe skill boundary. This produces an inspectable path from physical intervention to memory revision and changed control without REMM-specific parameter training.

We evaluate REMM on real-robot tasks involving transport disruptions, delayed sequence recall, and hidden-object rearrangement. The evaluation separates *retention* from *revision*: sequence and hidden-object tasks test whether decision-relevant history persists through delay and occlusion, while transport interventions test whether previously valid task state is correctly withdrawn and repaired after the world changes. Beyond end-to-end task success, we measure change acceptance, dependency revision, recovery timing and destination, physical repair, and the observed memory-to-control trace. Across 51 physical trials, REMM improves mean checkpoint progress over the strongest baseline from 57% to 91% in intervention recovery and from 43% to 70% in sequence recall, while completing all hidden-object trials.

Our contributions are threefold:

- a formulation of *revisable embodied memory* that separates recorded execution history from currently supported task state and represents intervention-sensitive dependencies from physical relations to derived and completed task conditions;
- REMM, a supervisory system that propagates dependency invalidation and dispatches nominal or recovery instructions to a frozen G0.5 policy without REMM-specific parameter training; and
- a controlled real-robot evaluation that traces observed changes through memory revision to subsequent control decisions across transport, sequence-recall, and hidden-object manipulation tasks.

II. RELATED WORK

A. Memory for Vision–Language–Action Control

Policy-integrated approaches make history part of action generation. MemoryVLA retrieves perceptual and cognitive tokens from a learned memory bank [2]; EventVLA learns when to preserve sparse visual evidence [8]; and Chameleon and ChainVLA propagate compact event or execution state across action queries [19], [20]. These methods learn representations for retention and retrieval inside the policy. RoboMME shows that their benefit depends strongly on whether a task requires temporal, spatial, object, or procedural memory [1]. REMM uses a different boundary: it leaves G0.5 frozen, does not learn a memory-conditioned action model, and studies explicit revision of task state between bounded skills.

Hierarchical approaches place memory above the executor. MemER trains a high-level visual policy to select keyframes and predict subtasks for a separately trained $\pi_{0.5}$ [9]. HiMe divides control among an Executor, a working-memory Sentry, and a Planner that retrieves and edits multimodal memory [10]. HyMeS uses executable memory-management heuristics and multimodal completion checks to steer a Markovian VLA [11], while BATON uses a frozen VLA with subtask exploration and transition-aware memory [12]. These systems are the closest architectural context for REMM. Our question is narrower than general memory management: whether storing explicit parents for derived and completed task conditions allows a verified intervention to revise exactly the downstream obligations that no longer hold.

B. Closed-Loop State Revision and Recovery

Robotic planners have long used feedback to reconsider high-level action. Inner Monologue conditions an LLM planner on success detection, scene description, and human feedback [13]; $\mathcal{P}^3$ schedules emerging tasks using priorities, dependencies, interruption, and resumption [21]; and PhyAgentOS couples persistent session state with verification and replanning [22]. These works establish closed-loop supervisory control, but their task scheduling or model-mediated replanning does not by itself expose which physical fact sustained a completed condition. REMM connects these levels explicitly and preserves the active atomic skill while inserting, deferring, or retargeting a recovery at a skill boundary.

Structured memories provide the complementary world-state representation. ReMemBR indexes spatially and temporally grounded experience for navigation [23]; ConceptGraphs builds open-vocabulary object–relation graphs for planning [14]; and RoboMemory revises relevant neighborhoods of temporal, spatial, semantic, and episodic memory [15]. REMM does not claim the first relational memory or editable knowledge graph. It adds an append-only evidence ledger, an active projection, and typed links from physical relations to derived scene and task conditions.

Those links draw on two related traditions. Truth-maintenance systems retain justifications and withdraw conclusions when their support becomes inactive [16]. Causal manipulation work instead uses controlled interventions or causal Bayesian models to reason about action effects [17], [18]. REMM does not discover a causal graph or predict general dynamics. It instantiates task-semantic causal dependencies whose operational meaning is intervention-sensitive: changing a required parent relation changes the validity of its descendant task condition and, through the controller, the selected continuation. The experiments therefore test both the stored revision and its downstream control effect rather than treating an edited memory record as sufficient evidence of recovery.

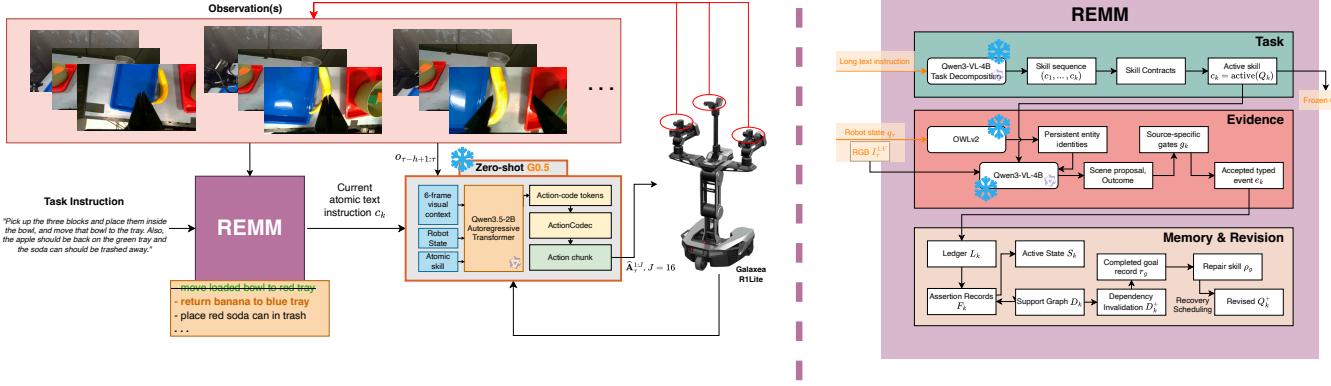


Fig. 2. **System and memory architecture.** Left: REMM converts a long-horizon instruction and multi-view observations into the current atomic instruction for a frozen G0.5 executor, which predicts action-code tokens and action chunks for the Galaxia R1 Lite; new observations and robot state close the loop after execution. Right: a task decomposer and skill contracts define the active skill, frozen perception and visual verification produce accepted typed events, and an append-only ledger maintains an active-state projection and support graph. When new evidence superseded a supporting fact, dependency invalidation withdraws affected goal records before REMM schedules a repair skill or issues the revised next query. Snowflakes denote frozen models.

III. METHOD

A. Problem Setting and System Overview

The robot receives episode instruction ℓ and observations $o_\tau = (I_\tau^{1:V}, q_\tau)$, comprising synchronized images from V views and proprioception. At each policy query τ , the frozen G0.5 executor consumes its latest h observations, REMM-selected atomic instruction c_k , and R1 Lite embodiment identifier e_{R1} :

$$(y_\tau, \hat{A}_\tau^{1:J}) = \pi_{G0.5}(o_{\tau-h+1:\tau}, c_k, e_{R1}; \theta_0), \quad \theta_0 \text{ fixed.} \quad (1)$$

The outputs are optional grounded reasoning tokens y_τ and a continuous action chunk $\hat{A}_\tau^{1:J}$. In our deployment, the client executes $J = 16$ actions at 15 Hz. REMM holds c_k fixed during the skill and selects a new instruction only at a verified boundary; the executor parameters θ_0 remain frozen.

At boundary k , REMM maintains $M_k = (L_k, F_k, D_k, Q_k)$: the append-only event ledger L_k , assertion records F_k , directed support graph D_k , and ordered skill queue Q_k . The currently dispatched instruction is $c_k = \text{active}(Q_k)$.

Separate from the G0.5 executor, REMM uses frozen Qwen3-VL-4B-Instruct [24] for constrained task decomposition and visual verification, and OWLv2-base-patch16-ensemble [25] for open-vocabulary 2D object proposals. Deterministic validators and source-specific evidence gates convert these outputs into accepted typed events; T2 and T3 use task-specific memory-query compilers rather than open-ended Qwen planning.

B. Revisable Memory State

Let $\varphi_i \equiv [p_i(\mathbf{x}_i) = v_i]$ be the grounded proposition in assertion record r_i , where p_i is a predicate, $\mathbf{x}_i$ contains its entity arguments, and $v_i \in \{\text{true}, \text{false}, \text{unresolved}\}$. Each record also stores confidence γ_i , evidence references $\mathcal{E}_i$, direct parents $\mathcal{P}_i$, and status $z_i \in \{\text{active}, \text{superseded}, \text{invalidated}\}$:

$$r_i = (\varphi_i, \gamma_i, \mathcal{E}_i, \mathcal{P}_i, z_i), \quad (2)$$

$$S_k = \{\varphi_i \mid r_i \in F_k, z_i = \text{active}\}.$$

Each $r_j \in \mathcal{P}_i$ creates edge $r_j \rightarrow r_i$ in D_k . The record set F_k preserves active and inactive assertions, while S_k projects only the propositions REMM currently treats as supported.

A candidate event e_k contains a proposed state change and its provenance. Binary gate g_s applies the acceptance rule for source $s = \text{src}(e_k)$; only accepted evidence enters the ledger:

$$L_{k+1} = \begin{cases} L_k \parallel e_k, & g_{\text{src}(e_k)}(e_k) = 1, \\ L_k, & \text{otherwise,} \end{cases} \quad (3)$$

where $\parallel$ denotes append. Visual relations require grounded identities, an allowed relation, noncontradictory evidence, confidence of at least 0.8, and two agreeing observations. Operator-verified outcomes and geometric contradictions use separate gates; a missing detection alone never retracts a fact.

C. Dependency Revision and Recovery

REMM separates direct physical assertions from their consequences. Let o be an object, c a container or support, l a coarse task region, and $R \in \{\text{inside}, \text{on}\}$. If active records place o relative to c and c at l , REMM derives the supported location of o :

$$r_s : R(o, c) = \top, \quad R \in \{\text{inside}, \text{on}\},$$

$$r_l : \text{at}(c, l) = \top,$$

$$r_s, r_l \implies r_d : \text{at}(o, l) = \top,$$

$$\mathcal{P}_d = \{r_s, r_l\}, \quad \gamma_d = \min(\gamma_s, \gamma_l).$$

Here l is a symbolic workspace region. The derived record stores both parents and inherits the weaker confidence, creating an explicit dependency that can later be withdrawn [26], [27].

For atomic placement goal g , $\text{verify}_k(g) = 1$ denotes an accepted physical postcondition at boundary k . REMM

TABLE I

CONTRIBUTION-ALIGNED EVALUATION COVERAGE. A CHECK INDICATES THAT AT LEAST ONE PREDECLARED CONDITION IN THE TASK FAMILY TESTS THE MECHANISM. T2 COMBINES NOMINAL ORDERED RECALL WITH DISTURBED REPEATS; ONLY THE DISTURBED REPEATS EXERCISE REVISION AND RECOVERY.

Family	Condition	History retention	Relation revision	Dependency invalidation	Recovery scheduling	Destination retargeting	Ordered recall	Identity continuity
T1	T1a: disruption with queued work	✓	✓	✓	✓	×	×	×
T1	T1b: destination changes after pouring	✓	✓	✓	✓	✓	×	×
T2	T2: ordered recall with disturbed repeats	✓	✓	✓	✓	×	✓	×
T3	T3: single hidden-object rearrangement	✓	×	×	×	×	×	✓

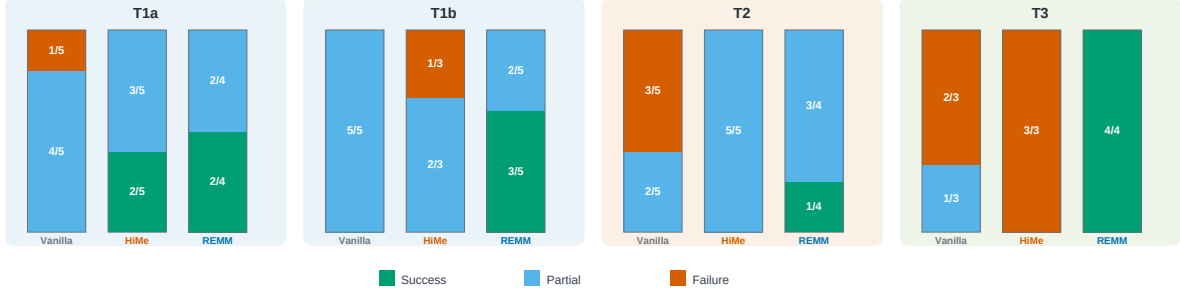


Fig. 3. **Physical-trial outcome decomposition.** Each task panel reports full success, partial completion, and failure for Vanilla, HiMe, and REMM. Raw counts preserve the task-specific denominators; panel backgrounds follow the transport, sequence, and hidden-object colors used throughout the paper.

represents completion as a derived record supported by the verified physical relation:

$$\begin{aligned}
 \text{verify}_k(g) &= 1, \quad R \in \{\textit{inside}, \textit{on}, \textit{at}\}, \\
 r_s : R(o, d) &= \top \\
 \implies r_g : \textit{satisfied}(g) &= \top, \\
 \mathcal{P}_g &\supseteq \{r_s\}.
 \end{aligned} \tag{5}$$

Goal completion therefore remains linked to the observed physical condition that justified it. This follows the general principle of semantic execution monitoring—checking expected action effects against observations [28]—while REMM additionally records the supporting fact.

Edges in D_k encode declared support relations, related to causal-link planning and task decomposition [29]; REMM supplies these links from task semantics rather than learning them from trajectories. Let D_k^+ denote reachability through one or more support edges. When record r_i becomes inactive, REMM invalidates every active descendant r_j :

$$\begin{aligned}
 z_i &\neq \text{active}, \quad (r_i, r_j) \in D_k^+, \\
 z_j &= \text{active} \implies z_j \leftarrow \text{invalidated}.
 \end{aligned} \tag{6}$$

As in truth maintenance, this operation preserves the evidence history while withdrawing only conclusions supported by the changed relation [16]. If invalidation reaches completed-goal record r_g , REMM creates or updates repair skill ρ_g . Let $E_g = \text{ent}(\rho_g)$ and $E_k = \text{ent}(c_k)$ contain the persistent entities involved in the repair and active instruction. Their overlap determines the queue update:

$$Q_k^+ = \begin{cases} \text{insert}_{\text{next}}(Q_k, \rho_g), & E_g \cap E_k \neq \emptyset, \\ \text{append}(Q_k, \rho_g), & \text{otherwise.} \end{cases} \tag{7}$$

Here Q_k^+ is the post-revision queue. The operator $\text{insert}_{\text{next}}$ places the repair immediately after the active skill, whereas append defers it behind the remaining work. Both operations deduplicate equivalent repairs and may retarget a pending repair using newer evidence. The active c_k is unchanged; the controller dispatches from Q_k^+ only at a stationary skill boundary.

IV. EXPERIMENTS

We evaluate REMM across 51 physical robot trials and 42 matched generated-video clips. Our experiments address three questions:

- 1) **Revision and recovery:** When new physical evidence contradicts a previously satisfied task condition, can REMM invalidate the dependent state and schedule the correct repair?
- 2) **Retention and grounding:** Can REMM preserve ordered and object-identity information through delay, occlusion, and rearrangement, then ground that history into the correct next instruction?
- 3) **Control effect and cost:** With the G0.5 executor held fixed, how do these memory decisions affect task progress, end-to-end success, failure modes, and system cost relative to no-memory and hierarchical-memory baselines?

A. Task Families

We organize the study by memory demand, rather than by atomic motor skill, as summarized in Fig. 4.

Transport and recovery tests deferring a correction while unrelated work remains queued (T1a) and retargeting recovery after a destination change (T1b).

Sequence recall (T2) tests recording a hidden demonstration order, grounding each historical cue to a current object, and compiling an ordered multi-skill queue; predeclared disturbed repeats additionally test whether completed placements are revised and repaired without corrupting the remaining order.

Hidden-object search (T3) instead tests whether one object–cover binding remains attached to the same cover through a shuffle.

B. Dependency Mechanism Controls

We isolate stored dependency invalidation from observing the same physical change. Let $\mathcal{V}_k$ contain completed-goal records newly invalidated at boundary k , let $h_k(r) = 1$ when support record r agrees with the latest direct facts, and let $b_k^\sigma(g)$ indicate that strategy σ declares goal g broken:

$$b_k^\sigma(g) = \begin{cases} \mathbb{I}[r_g \in \mathcal{V}_k], & \sigma = \text{explicit}, \\ \mathbb{I}[\exists r \in \mathcal{P}_g : h_k(r) = 0], & \sigma = \text{recompute}, \\ 0, & \sigma = \text{direct}. \end{cases} \quad (8)$$

The explicit strategy reads REMM’s stored invalidation, re-computation checks the saved postconditions on demand, and direct update changes only the physical relation. All three receive the same accepted evidence; this is a mechanism control, not an external baseline.

C. Evaluation Metrics

For trial e , let $\mathcal{G}_e = (g_{e,1}, \dots, g_{e,K_e})$ be evaluator-defined task checkpoints fixed before execution, and let $y_{e,m} \in \{0, 1\}$ indicate whether checkpoint m is independently satisfied. We report both partial task progress P_e and strict end-to-end success S_e :

$$\begin{aligned} P_e &= \frac{1}{K_e} \sum_{m=1}^{K_e} y_{e,m}, \\ S_e &= \prod_{m=1}^{K_e} y_{e,m}. \end{aligned} \quad (9)$$

Checkpoints are fixed before execution and shared across methods; they never use a controller’s self-generated plan. For disturbed trials, accepted change, invalidated task state, recovery placement in the queue, and physical repair are

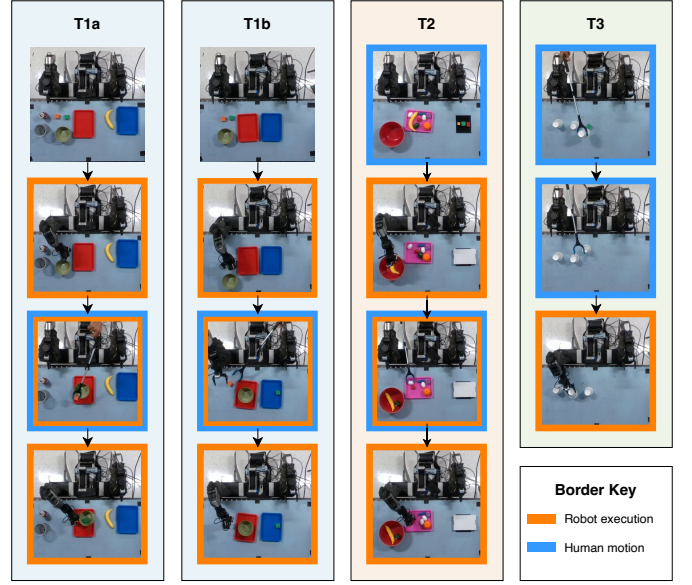


Fig. 4. **Physical evaluation setups.** T1a and T1b test recovery scheduling and retargeting. T2 tests ordered recall and disturbed execution, whereas T3 tests object–cover identity through rearrangement.

labeled separately so that state revision is not conflated with motor success.

$$\begin{aligned} B_w &= \sum_{i=1}^{N_w} |\text{serialize}(W_i)|, \\ B_r &= \sum_{j=1}^{N_r} |\text{serialize}(R_j)|, \\ \bar{n}_w &= N_w/N_q, \quad \bar{n}_r = N_r/N_q, \\ \bar{b}_w &= B_w/N_q, \quad \bar{b}_r = B_r/N_q. \end{aligned} \quad (10)$$

Here W_i and R_j are method-native logical write and retrieval payloads, and N_q counts fresh G0.5 action-chunk queries. Current observations, audit recordings, and model weights are reported separately rather than counted as memory traffic.

V. RESULTS

We report three complementary views: physical outcome decomposition, task-level decision and cost profiles, and matched offline diagnostics of high-level memory readout. Every rate uses independently labeled eligible trials as its denominator; incomplete or excluded runs are reported separately rather than counted as failures.

A. End-to-End Physical Outcomes

Figure 3 separates full task success from partial completion and failure. REMM is the only system with a full completion in every task family: 10 of 17 trials overall, compared with 2 of 16 for HiMe and 0 of 18 for vanilla G0.5. None of the 17 REMM trials is classified as a complete failure, although seven remain partial. These pooled counts are descriptive rather than a single comparable success rate because the task mix and denominators differ; the task-separated checkpoints below provide the more informative comparison.

Rates: observed %. Efficiency: 100 x best task median / method median.

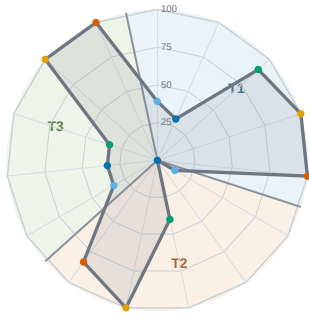
Progress

Decision

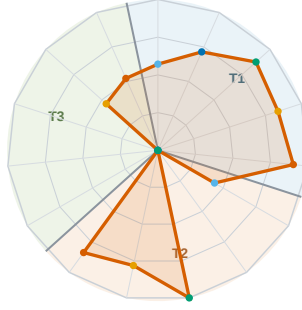
Completion

Time eff.

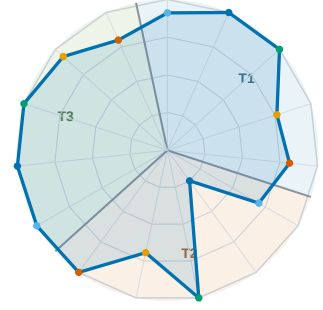
Query eff.



Vanilla G0.5 (n=18)



HiMe (n=16)



REMM (n=17)

Fig. 5. **Task-separated system profiles.** Each method has 15 vertices: checkpoint progress, decision accuracy, non-failure completion, time efficiency, and G0.5-query efficiency repeated within the colored T1, T2, and T3 sectors. T1 pools T1a and T1b. Higher is better. Rate denominators exclude blank labels; efficiency uses the proportional task-specific median-cost normalization defined in the text. Strict full-task success remains in Fig. 3.

TABLE II

MATCHED OFFLINE GENERATED-VIDEO DIAGNOSTIC. ALL METHODS RECEIVE THE SAME 12 T2 AND 30 T3 CLIPS. PERCENTAGES INCLUDE RAW COUNTS; THE FINAL COLUMN REPORTS T3 CUP CHOICE / GROUNDED LIFT COMMAND.

Method	T2 cue LCS	Exact seq.	First object	Destination	Grounded command	T3 cup / command
Vanilla G0.5 (no memory)	52.8% (19/36)	8.3% (1/12)	25.0% (3/12)	91.7% (11/12)	16.7% (2/12)	36.7% (11/30) / 36.7% (11/30)
HiMe	97.2% (35/36)	91.7% (11/12)	8.3% (1/12)	66.7% (8/12)	8.3% (1/12)	33.3% (10/30) / 33.3% (10/30)
REMM	97.2% (35/36)	91.7% (11/12)	91.7% (11/12)	83.3% (10/12)	66.7% (8/12)	100.0% (30/30) / 100.0% (30/30)

B. Task-Level System Profiles

Figure 5 reports checkpoint progress, decision accuracy, non-failure completion, execution-time efficiency, and G0.5-query efficiency separately for T1, T2, and T3; T1 pools T1a and T1b. The first three measures are observed percentages. For cost x , method m on task t receives efficiency $100 \min_{m'} \tilde{x}_{m',t} / \tilde{x}_{m,t}$, where $\tilde{x}_{m,t}$ is the median observed cost. This normalization preserves proportional cost differences while assigning 100 to the most efficient observed method per task.

The checkpoint profiles explain what the pooled outcomes hide. On T1, REMM raises mean progress from 57% for the strongest baseline to 91%; on T2, it raises progress from 43% to 70%; and on T3 it completes all four trials while neither baseline records a full completion. This gain carries a visible systems cost: REMM is not the most time- or query-efficient method on every task because it performs additional perception and verification and often continues runs that a baseline terminates early. We therefore report efficiency beside success rather than folding both into one score.

C. Offline Generated-Video Diagnostics

The generated-video comparison uses the same 12 independently reviewed T2 clips and 30 independently reviewed T3 clips for every method; representative frames appear in Fig. 6. T2 measures cue-order recall, exact three-item sequence, first-object and destination grounding, and a grounded first command; T3 requires one left, middle, or right cup choice and its corresponding lift command. The command scorer requires the

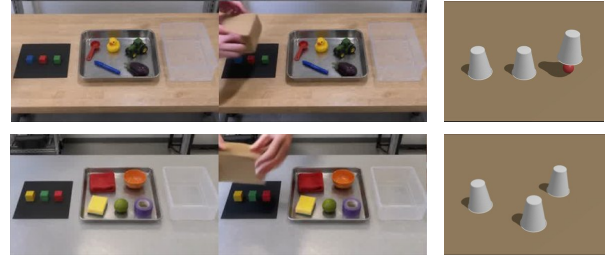


Fig. 6. **Generated-video diagnostics.** Left: T2 requires retaining a demonstrated color order after the cues are covered. Right: T3 requires preserving the hidden object's cover identity through a rearrangement. These diagnostics score high-level memory readout and command generation without executing the robot.

correct action, object or color, and destination while allowing a fixed set of transparent noun equivalences.

The T2 columns in Table II separate remembering a sequence from turning that memory into an actionable instruction. HiMe and REMM both recover 35 of 36 cue positions and 11 of 12 exact sequences, yet HiMe grounds the first object and command on only 1 of 12 clips, versus 11 of 12 and 8 of 12 for REMM. The sole offline column where vanilla exceeds REMM is destination identification (11 of 12 versus 10 of 12); this small reversal, alongside vanilla's weak sequence recall, indicates that recognizing the destination is not the principal memory bottleneck.

In T3, vanilla always chooses the middle cup and therefore matches the fixed-set majority, while HiMe produces all three

labels but remains at chance. REMM’s persistent target–cover track is correct on all 30 deterministic clips (Wilson 95% interval: 88.6–100%). This result is specific to the generated set and does not establish perfect tracking under real occlusion or appearance shift. These are high-level offline diagnostics rather than physical manipulation outcomes: passive video never executes G0.5, and observation selection and compute follow each method’s architecture.

VI. DISCUSSION

A. Interpreting the Results

The results support separating retention, revision, and grounding rather than treating embodied memory as a single capability. Nominal T2 and T3 test whether task-relevant information remains available after delay, covering, or rearrangement. T1 and the disturbed T2 repeats additionally require a previously supported task condition to become inactive. REMM is designed for this second case: accepted evidence is connected to the completed condition it supported, allowing a change to withdraw that condition and create a bounded repair instead of merely adding another observation to an undifferentiated history.

The offline T2 result clarifies where structured memory matters. HiMe recalls the demonstrated order nearly as well as REMM but rarely grounds that order to the correct first object or command. REMM’s improvement therefore coincides with persistent entity binding and the compilation of retained state into an atomic instruction, rather than with sequence storage alone. The comparison remains system-level: each method uses its native observation-selection and inference path, so the result does not attribute the difference to one isolated component or to memory size.

Holding G0.5 fixed removes policy training as a confound and makes the selected high-level instruction auditable, but it does not make every end-to-end failure a memory failure. The methods can differ in when they query observations, when they stop, and which instruction they send; the common executor can still fail to realize a correct instruction. We therefore interpret decision correctness, checkpoint progress, and physical success together. Likewise, the dependency controls isolate state-update logic in software, while the physical comparison measures complete systems; the physical gains should not be attributed to graph propagation alone.

B. Scope and Limitations

REMM’s dependency graph is supplied by the task compiler and a fixed predicate ontology. The recorded links have an intervention-sensitive causal interpretation within those declared task semantics: changing a required parent relation changes the validity of its dependent task condition. REMM does not discover causal structure from trajectories, predict arbitrary physical effects, or automatically compile unconstrained language into new dependency rules.

The live state estimate also remains bounded by perception and execution. It uses conservative 2D detections, persistent

identifiers, frozen VLM judgments, and source-specific evidence gates rather than calibrated metric 3D state. Missing detections do not retract facts, which reduces false revision but can delay necessary updates. Corrections are dispatched only at verified stationary skill boundaries; REMM does not interrupt an unsafe action chunk in progress or guarantee recovery when identity is lost under complete occlusion.

Finally, the physical study uses one robot, a small tabletop object set, controlled interventions, and limited per-condition samples. HiMe is evaluated as a controlled adaptation around the shared executor, so the comparison characterizes these deployed systems rather than every configuration of the original method. The generated clips provide matched diagnostic questions but remain deterministic and use method-native compute paths. Broader evaluation should test automatically constructed but auditable dependencies, calibrated 3D identity tracking, and safe revision during execution across additional robots, policies, and task families. To support reproducibility, we will release our implementation, evaluation protocols, controller configurations, structured memory traces, and full trial videos through the project website.

VII. CONCLUSION

Long-horizon manipulation requires a robot to retain useful history while withdrawing conclusions that later physical evidence no longer supports. REMM addresses this problem with an append-only evidence ledger, a revisable task-state projection, and explicit task-semantic dependencies that connect physical relations to completed conditions and bounded recovery instructions. Across 51 physical trials, REMM improves mean checkpoint progress on intervention-recovery tasks from 57% for the strongest baseline to 91%, while the matched diagnostics show that retaining a sequence is insufficient unless it can be grounded into the next action. These results support revisable memory as a practical supervisory interface for a frozen VLA, while leaving open the harder problems of learning task dependencies, maintaining identity under unconstrained perception, and revising control safely within an active skill.

REFERENCES

- [1] Y. Dai, H. Fu, J. Lee, Y. Liu, H. Zhang, J. Yang, C. Finn, N. Fazeli, and J. Chai, “RoboMME: Benchmarking and understanding memory for robotic generalist policies,” *arXiv preprint arXiv:2603.04639*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.04639>
- [2] H. Shi, B. Xie, Y. Liu, L. Sun, F. Liu, T. Wang, E. Zhou, H. Fan, X. Zhang, and G. Huang, “Memoryvla: Perceptual-cognitive memory in vision-language-action models for robotic manipulation,” *arXiv preprint arXiv:2508.19236*, 2025.
- [3] A. Brohan *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” *arXiv preprint arXiv:2307.15818*, 2023.
- [4] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees *et al.*, “Octo: An open-source generalist robot policy,” in *Robotics: Science and Systems*, 2024.
- [5] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “Open-VLA: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [6] Physical Intelligence *et al.*, “ $\pi_{0.5}$: A vision–language–action model with open-world generalization,” *arXiv preprint arXiv:2504.16054*, 2025.

- [7] Y. Liu, Z. Dong, B. Ye, T. Yuan, T. Jiang, A. Yang, S. Cao, H. Liu, Y. Sun, Z. Guo, X. Liu, D. Ke, C. Pan, C. Wu, T. Cheng, X. Ren, X. Zhang, J. Cui, Z. Zhao, H. Zhang, K. Xu, H. Yang, B. Zhang, J. Niu, S. Zhu, S. Zhang, and H. Zhao, "G0.5: One autoregressive stream for robot reasoning and action," 2026. [Online]. Available: <https://arxiv.org/abs/2608.11739>
- [8] G. Yang, Z. Tu, Y. Yang, S. Mao, J. Dong, T. Chen, J. Peng, J. Xiong, J. Cao, J. Dai, W. Zhou, Y. Mu, and T. Wang, "EventVLA: Event-driven visual evidence memory for long-horizon vision-language-action policies," *arXiv preprint arXiv:2606.20092*, 2026.
- [9] A. Sridhar, J. Pan, S. Sharma, and C. Finn, "MemER: Scaling up memory for robot control via experience retrieval," *arXiv preprint arXiv:2510.20328*, 2025.
- [10] L. Ji, S. Wang, P. Qian, X. Yu, Y. Tian, Z. Fei, J. Gong, and X. Qiu, "HiMe: Hierarchical embodied memory for long-horizon vision-language-action control," *arXiv preprint arXiv:2607.03449*, 2026.
- [11] Y. Zhao, Z. Ni, H. Chen, R. Zhang, and Q. Zhu, "Skills in weights, memory in code: Hybrid learning for memory-dependent robot manipulation," *arXiv preprint arXiv:2608.09410*, 2026.
- [12] B. Xu, Y. Shang, and E. Ferrara, "Don't drop the BATON: Long-horizon robot manipulation via agentic subtask exploration and transition-aware memory," *arXiv preprint arXiv:2608.16889*, 2026.
- [13] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng *et al.*, "Inner monologue: Embodied reasoning through planning with language models," *arXiv preprint arXiv:2207.05608*, 2022.
- [14] Q. Gu, A. Kuwajerwala, S. Morin, K. M. Jatavallabhula, B. Sen, A. Agarwal, C. Rivera, W. Paul, K. Ellis, R. Chellappa, C. Gan, C. M. de Melo, J. B. Tenenbaum, A. Torralba, F. Shkurti, and L. Paull, "ConceptGraphs: Open-vocabulary 3d scene graphs for perception and planning," in *2024 IEEE International Conference on Robotics and Automation*, 2024, pp. 5021–5028.
- [15] M. Lei, H. Cai, Y. Yang, Y. Wu, J. Ren, Z. Cui, L. Tan, J. Hong, G. Hu, S. Zhu, S. Jiang, G. Wang, J. Tan, Z. Wan, Z. Li, Z. Li, S. Cui, Y. Zhao, and Y. Han, "RoboMemory: A brain-inspired multi-memory agentic framework for interactive environmental learning in physical embodied systems," 2026. [Online]. Available: <https://arxiv.org/abs/2508.01415>
- [16] J. Doyle, "A truth maintenance system," *Artificial Intelligence*, vol. 12, no. 3, pp. 231–272, 1979.
- [17] O. Ahmed, F. Träuble, A. Goyal, A. Neitz, Y. Bengio, B. Schölkopf, M. Wüthrich, and S. Bauer, "CausalWorld: A robotic manipulation benchmark for causal structure and transfer learning," in *International Conference on Learning Representations*, 2021.
- [18] R. Cannizzaro, M. Groom, J. Routley, R. O. Ness, and L. Kunze, "COBRA-PPM: A causal bayesian reasoning architecture using probabilistic programming for robot manipulation under uncertainty," in *2025 European Conference on Mobile Robots*, 2025, pp. 1–8.
- [19] X. Guo, C. Jiang, H. B. Kim, Y. Han, Y. Sun, Y. Xiao, and J. Yang, "Chameleon: Control-indexed prospective memory for visuomotor manipulation," *arXiv preprint arXiv:2603.24576*, 2026.
- [20] Y. Huang, W. Bu, Z. Xiong, J. Wu, F. Huang, J. Jiang, and Z. Wang, "ChainVLA: Chaining vision-language-action queries through a unified execution state for long-horizon manipulation," *arXiv preprint arXiv:2608.02326*, 2026.
- [21] S. Zhou, X. Wang, J. Zhang, R. Tian, J. Huang, R. Xu, G. Chen, and F. Zheng, " $\mathcal{P}^3$: Toward versatile embodied agents," 2026. [Online]. Available: <https://arxiv.org/abs/2508.07033>
- [22] Y. Liu, W. Chen, X. Song, T. Pu, S. Mo, Y. Bai, Z. Chen, Q. Sun, L. Zhong, Y. Shen, and L. Lin, "PhyAgentOS: A self-evolving operating system for embodied agents with decoupled cognitive planning and physical execution," *arXiv preprint arXiv:2607.16636*, 2026.
- [23] A. Anwar, J. Welsh, J. Biswas, S. Pouya, and Y. Chang, "ReMEmbR: Building and reasoning over long-horizon spatio-temporal memory for robot navigation," *arXiv preprint arXiv:2409.13682*, 2024.
- [24] S. Bai, Y. Cai, R. Chen *et al.*, "Qwen3-VL technical report," *arXiv preprint arXiv:2511.21631*, 2025. [Online]. Available: <https://arxiv.org/abs/2511.21631>
- [25] M. Minderer, A. Gritsenko, and N. Houlsby, "Scaling open-vocabulary object detection," in *Advances in Neural Information Processing Systems*, 2023.
- [26] W. Liang, Y. Zhao, Y. Zhu, and S.-C. Zhu, "What is where: Inferring containment relations from videos," in *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence*, 2016, pp. 3418–3424.

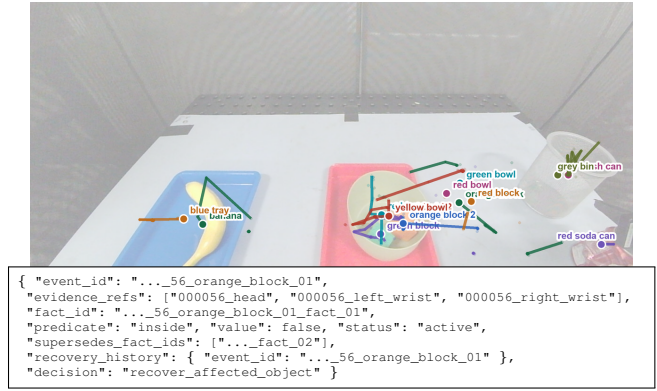


Fig. 7. **Recorded T1b memory example.** The trace contains 2,408 head-camera positions for 15 identities; gaps are disconnected tracks. The joined excerpt exposes evidence-to-event, fact-supersession, and recovery references. This assisted partial rollout uses the recompute-on-demand control and is included only for mechanism inspection.

- [27] M. Tenorth and M. Beetz, "Representations for robot knowledge in the KnowRob framework," *Artificial Intelligence*, vol. 247, pp. 151–169, 2017.
- [28] A. Bouguerra, L. Karlsson, and A. Saffiotti, "Monitoring the execution of robot plans using semantic knowledge," *Robotics and Autonomous Systems*, vol. 56, no. 11, pp. 942–954, 2008.
- [29] R. M. Young, M. E. Pollack, and J. D. Moore, "Decomposition and causality in partial-order planning," in *Proceedings of the Second International Conference on Artificial Intelligence Planning Systems*, 1994, pp. 188–193.

APPENDIX A

MEMORY RECORDS AND UPDATE SEMANTICS

A. Stored Records and References

Accepted observations become immutable events containing entity identifiers, predicate changes, confidence values, and frame references. Events create revisable facts, while sub-goal records retain completion and recovery evidence. The `supersedes_fact_ids` and `depends_on_fact_ids` fields trace each update from its visual evidence to the affected goal and repair.

B. Boundary Updates and Recovery

A relation is accepted when identities are grounded, confidence reaches 0.8, and two observations agree; missing detections alone do not create negative facts. At stationary boundaries, accepted events supersede conflicts, invalidate unsupported conclusions, and dispatch deduplicated repairs to G0.5. Figure 7 shows this process in T1b; the assisted partial rollout is included only for mechanism inspection.